

Question 1

You are given a **3D array of size $X \times Y \times Z$** , filled with integer values representing the scores of students in different subjects across multiple campuses. Your task is to **sort the entire 3D array in ascending order** while ensuring efficient memory management.

Write a **function** to dynamically allocate a **3D array** and another function to **fill the array** with values. Implement a **function to sort the entire 3D array** in ascending order.

After sorting, implement separate **functions** to compute the following statistical values:

- **Average (Mean):** Compute the sum of all elements divided by the total count.
- **Sum:** Calculate the total sum of all elements.
- **Median:** Determine the middle value(s) in the sorted array.
- **Mode:** Find the most frequently occurring value(s) in the array. If no mode exists (i.e., all values appear the same number of times), print "NO mode found".

Ensure that memory is dynamically allocated and properly deallocated to prevent memory leaks.

Question 2

The Hospital class manages an array of Patient objects and an array of Doctors objects dynamically. It should contain:

- Private attributes
 - Patient* patients: Dynamic array of patients.

- Doctor* doctors: Dynamic array of doctors.
- int patientCount: Number of patients.
- int doctorCount: Number of doctors.
- Constructors & Destructor:
 - Default Constructor: Initializes empty arrays for patients and doctors.
 - Parameterized Constructor: Takes initial arrays of Patient and Doctor objects and their sizes as input.
 - Copy Constructor: Performs a deep copy of another Hospital object.
 - Destructor: Properly deallocates the dynamically allocated memory for the arrays.
- Member Functions:
 - void addPatient(const Patient &p): Adds a new patient to the hospital.
 - void addDoctor(const Doctor &d): Adds a new doctor to the hospital.
 - void displayAllPatients(): Displays details of all patients.
 - void displayAllDoctors(): Displays details of all doctors.
 - void admitPatient(int patientId, int doctorId): Admits a patient and assigns them to a doctor (if the doctor has capacity).
 - void dischargePatient(int patientId): Discharges a patient and releases them from the doctor.
 - Patient* searchPatient(int patientId): Searches for a patient by ID and returns a pointer to the patient object (or nullptr if not found).
 - Doctor* searchDoctor(int doctorId): Searches for a doctor by ID and returns a pointer to the doctor object (or nullptr if not found).

- void sortPatientsByName(): Sorts the array of patients by name in alphabetical order.
- void sortDoctorsBySpecialization(): Sorts the array of doctors by specialization in alphabetical order.

Constraints:

- Use dynamic memory allocation for the arrays of patients and doctors.
- Ensure proper memory management to avoid memory leaks or dangling pointers.
- Handle edge cases, such as:
 - Invalid patient or doctor IDs (e.g., patient not found).
 - Exceeding a doctor's patient capacity when admitting a new patient.

Question 3

- Define a class Item with private data members:
 - string name (stores the name of the item).
 - string vendor (stores the name of the vendor).
 - double price (stores the selling price of the item).
 - double cost (stores the cost price of the item).
 - double weight (stores the weight of a single item).
 - bool taxable (indicates whether tax is charged on the item).
 - Implement a **constructor** that takes the item's name, cost, and price as arguments and initializes weight to 1 and taxable to true.
- Implement **getter functions** for name, vendor, price, weight, and taxable.
- Implement **setter functions** for weight and taxable.
- Implement a **void function** increaseCost() that increases the cost price by 5%.

- Implement a **function** `double profit()` that calculates and returns the profit (price - cost).
- Create an **array of multiple Item objects** (size determined by the user).
- Use a loop to:
 - Calculate and display the **total weight** of all items.
 - Find and display the **item with the highest price**.
 - Count and display the number of **taxable items (profitable items)**.
- Implement a **suitable main() function** that:
 - Demonstrate object creation using the **constructor**.
 - Calls **setter functions** to update weight and taxable status.
 - Calls **increaseCost()** to modify the cost price.
 - Calls **profit()** to display item profitability.
 - Processes an array of item objects to perform the required calculations.
- Ensure **separate function declarations and definitions** outside the class.